Robotics (ECE670/CSE633)

Course Project

Physics-Informed Learning for Robot Dynamics and Control

Course Instructor: Dr. Sayan Basu Roy

In the course project, your group chooses its own problem in physics-informed learning for robot dynamics and control. You will explore systems from the DeepMind Control Suite, such as : robotic arms and the pendulum on a cart (cart-pole), learn their dynamics with physics-informed neural networks, and compare these networks with black-box networks and with Euler–Lagrange models like the one you derive in Assignment 2. There is no mid-semester review: groups give a short pre-evaluation demo on the last day of classes, and the final evaluation (report, live demonstration, presentation and individual viva) follows the end-semester examinations.

Theme and Scope

Every project must contain the following four elements. Scope the project so that it can be finished in the time between approval and the final submission: one system, one physics-informed model, one baseline, and one closed-loop task are enough.

- **System.** A robotic arm or a pendulum-type system simulated with MuJoCo through the DeepMind Control Suite (`dm_control`); see Table 1. You may also load other MuJoCo models with `dm_control`, for example the ROBOTIS model of the OpenMANIPULATOR-X used in Assignments 1–3.
- **Physics-informed learning.** A neural network that uses the structure of the Euler–Lagrange equations, for example a physics-informed neural network (PINN) with an equation-of-motion residual loss, a Lagrangian or Hamiltonian neural network, or a Deep Lagrangian Network.
- **Baselines.** A comparison with a black-box neural network trained on the same data and with the analytical model of the system.
- **Task.** The learned model is used for prediction and in one closed-loop task in simulation, for example computed-torque tracking, swing-up and balance, or model-predictive control.

Groups and Project Topic

- The course project sheet, `Robotics_2026_Course_Project_Group_Number`, is linked on Google Classroom. Enter your group and your project statement there.
- Form groups of 2 or 3 students and enter the group details by **Thursday, 29 October 2026**.
- Each group identifies its own topic: the system, the physics-informed approach, and the task. The statement must be technical enough to be evaluated, with clearly identified inputs, outputs, and measures of success.
- Enter the project statement within one week, by **Monday, 2 November 2026**. The instructor must approve each statement; approval is communicated in the sheet by **Wednesday, 4 November 2026**.
- Two groups may not choose the same statement (the same system, approach, and task). If that happens, the statement goes to the group that entered it first, and the other group must find a different one.
- The following examples show the expected scope. Use them for orientation only and propose your own statement:
  - a Lagrangian neural network of the cart-pole, used for energy-based swing-up and balance;

– a Deep Lagrangian Network of the reacher arm, used for computed-torque tracking;
– physics-informed identification of the friction and rotor inertia in the MuJoCo model of the OpenMANIPULATOR-X, compared with the URDF model of Assignment 2;
– a Hamiltonian neural network for long-horizon prediction of the acrobot, with a study of energy drift.

Getting Started with the DeepMind Control Suite

Implement the project in Python with `dm_control` and PyTorch or JAX; Google Colab is acceptable. At present, `pip install dm_control` needs Python 3.10–3.12. Every suite task takes actions in $[-1, 1]$, and each actuator multiplies its action by a gear ratio to give the joint force or torque, $\tau = \text{gear} \times \text{action}$. Table 1 lists the systems that suit this project; `suite.ALL_TASKS` lists every domain and task.

Table 1. Systems in the DeepMind Control Suite that suit the project.

Domain (tasks)	Generalized coordinates	Actuation (gear)
`pendulum` (`swingup`)	Hinge angle	Torque (1), too weak to lift the pole directly
`cartpole` (`balance`, `swingup`, `two_poles`, `three_poles`)	Cart position and one to three pole angles	Force on the cart (10); the poles are unactuated
`acrobot` (`swingup`)	Shoulder and elbow angles	Elbow torque only (2)
`reacher` (`easy`, `hard`)	Two joint angles of a planar two-link arm	Two joint torques (0.05 each)
`manipulator` (`bring_ball`, `insert_peg`, ...)	Four arm joints, the gripper joints, and the object	Four arm torques (12, 8, 4, 2) and a gripper tendon; contacts
OpenMANIPULATOR-X (ROBOTIS model)	Four arm joints and the gripper	Position servos; switch to torque actuators

The script below collects one episode of training data from the cart-pole with random actions. `physics.forward()` recomputes the dynamics at the new state with the same action, so each row holds $(q, \dot{q}, \tau, \ddot{q})$, with τ taken from `qfrc_actuator`.

```
import numpy as np
from dm_control import suite                              # needs Python 3.10-3.12
env = suite.load("cartpole", "swingup", task_kwargs={"random": 0})
physics, spec = env.physics, env.action_spec()
rng = np.random.default_rng(0)
rows, ts = [], env.reset()
while not ts.last():                                      # one episode: 1000 steps of 0.01 s
    ts = env.step(rng.uniform(spec.minimum, spec.maximum))  # random action in [-1, 1]
    physics.forward()                                     # dynamics at the new state
    d = physics.data                                      # each row: q, q_dot, tau, q_ddot
    rows.append(np.concatenate([d.qpos, d.qvel, d.qfrc_actuator, d.qacc]))
data = np.array(rows)                                     # shape (1000, 4 * nv)
```

- `mujoco.mj_inverse`, called with `physics.model.ptr` and `physics.data.ptr`, writes the inverse dynamics of the current state to `physics.data.qfrc_inverse`; compare it with your analytical model. `physics.render(240, 320)` returns an RGB frame for figures and videos (set `MUJOCO_GL=egl` on a machine without a display).
- For the robot of Assignments 1–3, use the ROBOTIS model of the OpenMANIPULATOR-X in the GitHub repository `robotis_mujoco_menagerie`, folder `robotis_open_manipulator_x`. Load it with `Physics.from_xml_path` from `dm_control.mujoco`; it adds rotor armature and joint friction

to the URDF parameters of Assignment 2. Its actuators are position servos: for torque-level dynamics, replace them with motor actuators or apply joint torques through `physics.data.qfrc_applied`.

A. The Project Report

Submit the report as a PDF with these sections: Introduction; Problem Statement (clear and precise); Methodology; Results; Conclusion. Across these sections, the report must cover the following.

1. **Problem statement.** The system (`dm_control` domain and task, or MuJoCo model), its generalized coordinates, the actuated and unactuated joints, and how actions map to joint forces. State what the network learns, for example the forward dynamics $\ddot{q} = f(q, \dot{q}, \tau)$, the inverse dynamics, the energy, or unknown parameters, and why the physics should help.
2. **Analytical model.** The Euler–Lagrange equations of the system, as in Assignment 2, with parameters from the MuJoCo model (masses, inertias, damping, friction, armature, gear), validated against `mujoco.mj_inverse` along recorded trajectories.
3. **Data generation.** The excitation (random, sinusoidal, or chirp actions, or controller rollouts), state ranges, time step, dataset sizes, and any added measurement noise; training, validation, and test sets, including a test set outside the training range.
4. **Models.** (a) A black-box neural network baseline; (b) the physics-informed network: its architecture, how the physics enters (a residual loss on the Euler–Lagrange equations, a Lagrangian or Hamiltonian structure, a positive-definite inertia matrix, known parameters), the loss terms and weights, and the training settings and random seeds.
5. **Evaluation.** One-step prediction error; multi-step rollout error over a stated horizon; energy drift for unforced, undamped motion where applicable; data efficiency with at least three training-set sizes; generalization outside the training range; and identified parameters compared with the MuJoCo values, where applicable.
6. **Closed-loop task.** One controller in `dm_control` that uses the learned model, for example computed torque (as in Assignment 3), energy-based swing-up with balance, or model-predictive control, run with the analytical, black-box, and physics-informed models and the same metrics.
7. **Discussion.** When the physics prior helps, failure cases, computational cost, and limitations, and how your results relate to Assignments 2 and 3.

B. Evaluation (15 % of the Course Grade)

1. **Report — 30 %** formatting and quality , problem statement , analytical model and data , physics-informed model , results and comparison.
2. **Demonstration — 20 %.** Run the submitted code live: rollouts of the learned models against MuJoCo and the closed-loop task.
3. **Viva and Presentation — 50 %,** individual: each member answers questions on any part of the project.

C. Submission Guidelines

1. Upload the report, the code (scripts or notebooks with a `requirements.txt` and a `README` listing the commands that reproduce each figure), the trained model weights, the minutes of meetings, and a short demonstration video (or a link to it) in Google Classroom as a single zip file named `Robotics_project_group#no.zip`.
2. The report must be in PDF format; photographs of handwritten notes will not be accepted as a valid report.

3. Plots must be generated from your code; they cannot be mere screenshots. Marks will be deducted if plot labels, legends, and titles are not readable. Rendered frames from the simulator are acceptable as figures.
4. The submission will not be considered for evaluation if the naming format or the prescribed report format is not followed.

Final submission (by 23:59): After the end-semester exams (date to be announced). **Final presentation** (presentation, demonstration and individual viva): Before the course moderation (dates to be announced); slots will be published in the course project sheet.

Timeline

Table 2. Project timeline. The two internal milestones are suggestions and are not graded.

Milestone	**Date**
Group details entered in the course project sheet	Thursday, 29 October 2026
Project statement entered (within one week)	Monday, 2 November 2026
Approval communicated in the sheet	Wednesday, 4 November 2026
Suggested: environment, analytical model, and data pipeline ready	Thursday, 12 November 2026
Last day to request a meeting with the instructor	Friday, 13 November 2026
Suggested: physics-informed and baseline networks trained	Wednesday, 18 November 2026
Pre-evaluation: short demo with a few slides, in class	**Monday, 23 November 2026**
Final submission by 23:59: report, code, model weights, video, minutes of meetings	**After the end-semester exams (date to be announced)**
Final presentation: presentation, demonstration and individual viva	**Before the course moderation (dates to be announced)**

Meetings and Academic Integrity

- As a group, you may ask for two meetings with the instructor to understand and clarify the design requirements. Before these meetings, put a substantial amount of thought into your problem statement. Meeting requests after Friday, 13 November 2026 will not be entertained.
- Although it is a group project, marks will be assigned individually: a member whose contribution or understanding is found to be unsatisfactory will receive significantly fewer marks. Cite all external code, models, and datasets you use; the institute's academic dishonesty and plagiarism policy will be strictly enforced.